DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SUBJECT NAME: LANGUAGE TRANSLATORS **SUBJECT CODE: CST54**

UNIT- III

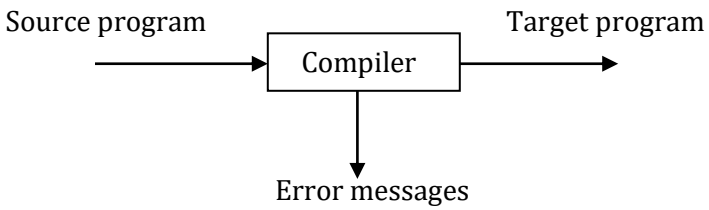
Source Program Analysis: Compilers – Analysis of the Source Program – Phases of a Compiler –Cousins of Compiler – Grouping of Phases – Compiler Construction Tools.

Lexical Analysis: Role of Lexical Analyzer – Input Buffering – Specification of Tokens – Recognition of Tokens –A Language for Specifying Lexical Analyzer.

2 MARKS

1. What is a compiler? **(MAY, NOV 2012)**

A *compiler* is a program that reads a program written in one language – the source language and translates it into an equivalent program in another language – the target language. In translation process, the compiler reports to its user the presence of errors in the source program.



2. What are the classifications of a compiler?

Compilers are sometimes classified as:

- Single-pass
- Multi-pass
- Load-and-go
- Debugging or
- Optimizing

3. What are the two parts of a compilation? Explain briefly.

There are two parts to compilation as

1. Analysis part
2. Synthesis part

- The *analysis part* breaks up the source program into constituent pieces and creates an intermediate representation of the source program.
- The *synthesis part* constructs the desired target program from the intermediate representation.

4. What are the tools available in analysis phase?

Many software tools that manipulate source programs are

- Structure editors
- Pretty printers
- Static checkers
- Interpreters

5. What is meant by Static Checking? (MAY 2014)

A static checker reads a program, analyzes it, and attempts to discover potential bugs without running the source program.

6. What is Query Interpreters?

A *Query interpreter* translates a predicate containing relational and Boolean operators into commands to search a database for records satisfying that predicate.

7. List the analysis of the source program?

Analysis consists of three phases:

1. Linear Analysis
2. Hierarchical Analysis
3. Semantic Analysis

8. What is linear analysis or lexical analysis? (MAY 2016)

- In a compiler, *linear analysis* is called *lexical analysis* or *scanning.*
- *Linear analysis* in which the stream of characters making up the source program is read from left-to-right and grouped into tokens that are sequences of characters having a collective meaning.

9. What is hierarchical analysis? (NOV 2011)

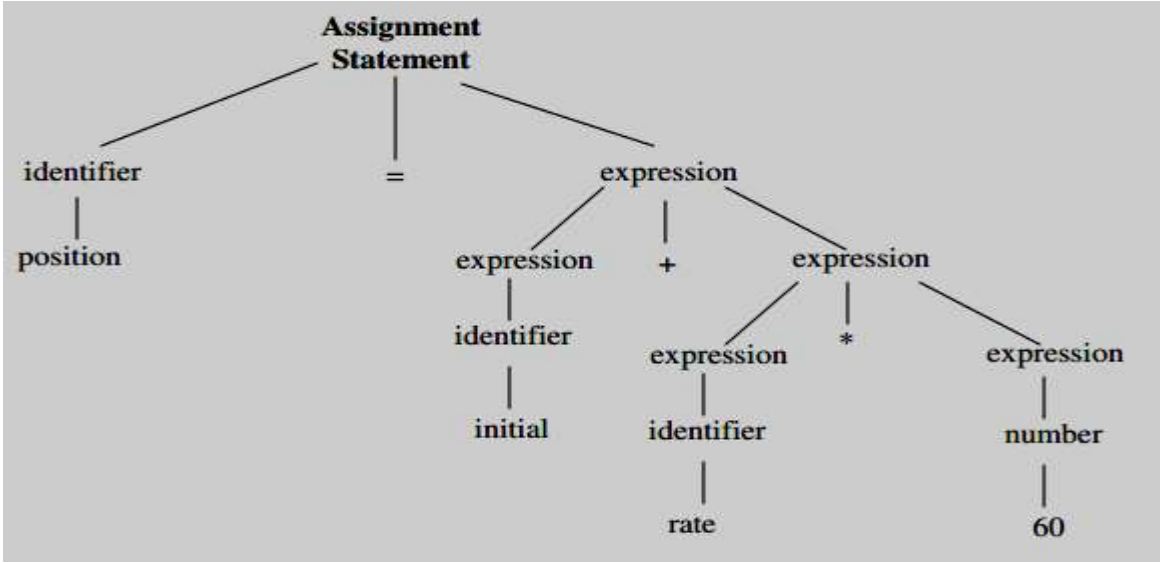
- *Hierarchical analysis* is called *parsing* or *syntax analysis*.

- *Hierarchical analysis* involves grouping the tokens of the source program into grammatical phases that are used by the compiler to synthesize output.

10. What is semantic analysis?

- The *Semantic analysis*, it checks the source program for semantic errors and gathers type information for the subsequent code generation phase.
- It uses the hierarchical structure determined by the syntax analysis phase to identify the operators and operands of expressions and statements.

11. Draw the parse tree for a source program as position: = initial + rate * 60.



12. List the various phases of a compiler?

The following are the various phases of a compiler:

- Lexical Analyzer
- Syntax Analyzer
- Semantic Analyzer
- Intermediate code generator
- Code optimizer
- Code generator

13. What is a symbol table?

- A *symbol table* is a data structure containing a record for each identifier, with fields for the attributes of the identifier. The data structure allows us to find the record for each identifier quickly and to store or retrieve data from that record quickly.
- Whenever an identifier is detected by a lexical analyzer, it is entered into the symbol table. The attributes of an identifier cannot be determined by the lexical analyzer.

14. What is Intermediate code generator?

The intermediate representation should have two important properties;

- it should be easy to produce, and
- it should be easy to translate into the target program

15. Define three address codes? **(NOV 2016)**

- An intermediate form called "*three-address code*, "which is like the assembly language for a machine in which every memory location can act like a register.
- Three-address code consists of a sequence of instructions, each of which has at most three operands.

16. What is code generator? **(MAY 2018)**

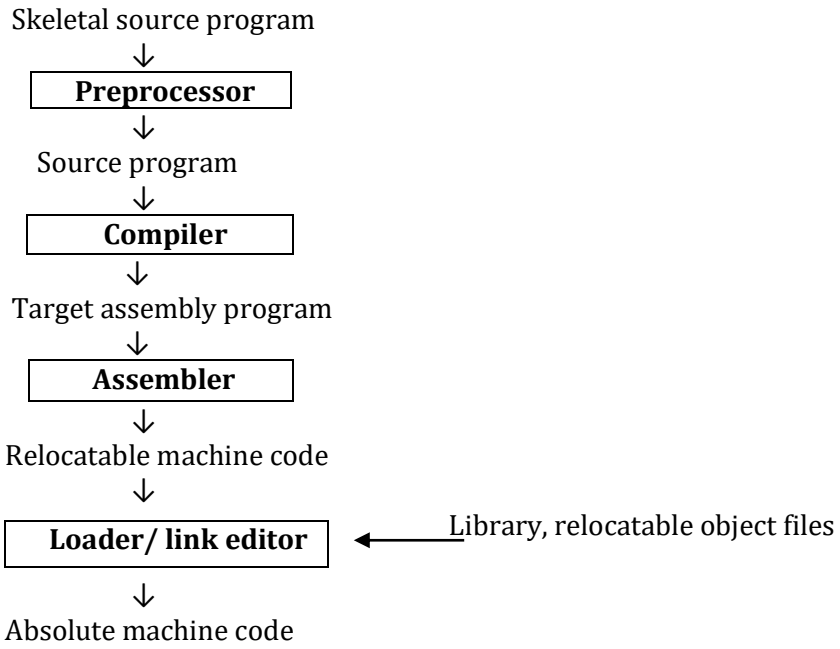
- The final phase of the compiler is the generation of target code, consisting normally of relocatable machine code or assembly code.
- Memory locations are selected for each of the variables used by the program. Then, intermediate instructions are each translated into a sequence of machine instructions that perform the same task.

17. Mention the cousins of the compiler?

Cousins of the compiler are:

- Preprocessor
- Compiler
- Assembler
- Two-Pass Assembly
- Loader and Link-Editor

18. State with example the cousins of compilers. **(NOV 2013)**



19. What is Preprocessors? List its functions.

Preprocessors produce input to compilers. They may perform the following functions:

- Macro Processing
- File inclusion
- Rational Preprocessors
- Language extensions

20. Define Assembly code?

- Assembly code is a mnemonic version of machine code, in which names are used instead of binary codes for operations, and names are also given to memory addresses.
- A typical sequence ***b := a + 2***,the assembly instructions might be

MOV a, R1

ADD #2, R1

MOV R1, b

21. What is the use Two-Pass assembly?

The assembler makes two passes over the input, where a *pass* consists of reading an input file once.

- In the *first pass*, all the identifiers that denote storage locations are found and stored in a symbol table.
- Identifiers are assigned storage locations as they are encountered for the first time.
- In the *second pass*, the assembler scans the input again. This time, it translates each operation code into the sequence of bits representing that operation in machine language and it translates each identifier representing a location into address given for that identifier in the symbol table.
- The output of the second pass is usually relocatable machine code.

22. What is loader and link-editor?

- Usually, a program called a loader performs the two functions of *loading* and *link-editing.*
- The process of *loading* consists of taking relocatable machine code, altering the relocatable addresses, and placing the altered instructions and data in memory at the proper location.
- The *link-editor* allows us to make a single program from several files of relocatable machine code. These files may be the result of several different compilation and one or more library files of routines provided by the system.

23. State the function of front end and back end of a compiler phase. **(MAY 2013)**

The front end consists of those phases that depends primarily on the source language and are largely independent of the target machine.

These includes

- Lexical analysis
- Syntactic analysis
- Semantic analysis
- Creation of symbol table

- Generation of intermediate code
- Code optimization
- Error handling

24. State the function back end of a compiler phase. **(MAY 2013)**

The back end of compiler includes those portions that depend on the target machine and generally those portions do not depend on the source language, just the intermediate language.

These include

- Code optimization
- Code generation
- Error handling and
- Symbol-table operations

25. What is single pass?

Several phase of compilation are usually implemented in a single pass consisting of reading an input file and writing an output file.

26. Define compiler-compiler.

Systems to help with the compiler-writing process are often been referred to as *compiler-compilers, compiler-generators or translator-writing systems.* Largely they are oriented around a particular model of languages, and they are suitable for generating compilers of languages similar model.

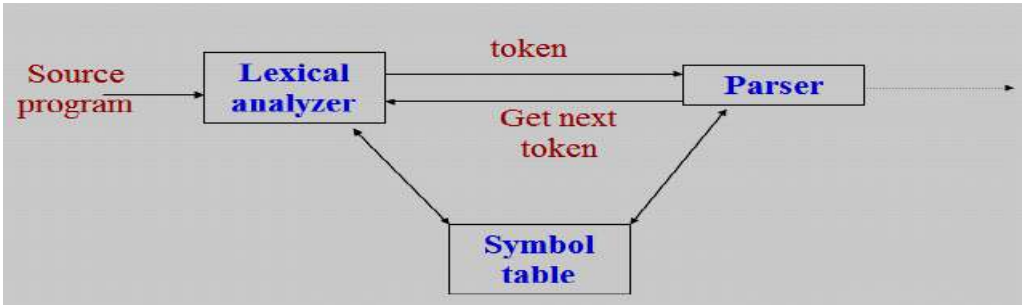
27. List the various compiler construction tools.

The various compiler construction tools are

1. Parser generators
2. Scanner generators
3. Syntax-directed translation engines
4. Automatic code generators
5. Data-flow engines

28. What is the role of lexical analyzer? **(NOV 2013, 2015) (MAY 2018)**

- The *lexical analyzer* is the first phase of a compiler.
- Its main task is to read the input characters and produce as output a sequence of tokens that the parser uses the syntax analysis.
- Upon receiving a "get next token" command from the parser, the lexical analyzer reads input characters until it can identify the next token.



29. What are the issues in lexical analysis?

The issues in lexical analysis are

- Simpler design is the most important consideration.
- Compiler efficiency is improved.
- Compiler portability is enhanced.

30. Why separate lexical analysis phase is required? **(MAY 2013)**

i. Simpler design is the most important consideration.
- Comments and white space have already been removed by lexical analyzer.

ii. Compiler Efficiency is improved.
- Specialized buffering techniques for reading input characters and processing tokens can significantly speed up the performance of a compiler.

iii. Compiler Portability is enhanced.
- Input alphabet peculiarities and other device-specific anomalies can be restricted to the lexical analyzer.

31. What is the major advantage of a lexical analyzer generator? **(NOV 2011)**

The major advantages of a lexical analyzer generator are

- One task is stripping out from the source program comments and white space in the form of blank, tab and new line characters.
- Another is error messages from the compiler in the source program.

32. What are tokens, patterns, and lexeme?

- *Tokens*- Sequence of characters that have a collective meaning.
- *Patterns*- There is a set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a pattern associated with the token.
- *Lexeme*- A sequence of characters in the source program that is matched by the pattern for a token.

33. Differentiate between tokens, patterns, and lexeme?

Token	lexeme	patterns
const	const	const
if	if	if
relation	<, <=, =, < >, >, >=	< or <= or = or < > or >= or >
id	pi, count, D2	letter followed by letters and digits
num	3.1416, 0, 6.02E23	any numeric constant
literal	"core dumped"	any characters between " and " except "

34. List the various Panic mode recovery in lexical analyzer?

Possible error recovery actions are:

1. Deleting an extraneous character
2. Inserting a missing character
3. Replacing an incorrect character by a correct character
4. Transposing two adjacent characters

35. What are the approaches to implement lexical analyzer?

The three general approaches to the implementation of a lexical analyzer

1. Use a lexical analyzer generator such as the Lex compiler to produce a regular expression based specification. In this case, the generator provides for reading and buffering the input.
2. Write the lexical analyzer in a conventional systems programming languages using the I/O facilities of that language to read the input.
3. Write the lexical analyzer in assembly language and reading of input.

36. What is input buffering? **(MAY 2017)**

- The *input buffer* is useful when look-ahead on the input is to identify tokens.
- To ensure that a right lexeme is found, one or more characters have to be looked up beyond the next lexeme.
- Hence a two-buffer scheme is introduced to handle large look-ahead safely.
- Techniques for speeding up the process of lexical analyzer such as the use of sentinels to mark the buffer end have been adopted.

37. Define sentinels. **(MAY 2014)**

- *Sentinel* is a special character that cannot be part of the source program.
- The techniques for speeding up the lexical analyzer, use the "sentinels " to mark the buffer end.

38. List the operations on languages.

The operations that can be applied to languages are

- Union - L U M ={s | s is in L or s is in M}
- Concatenation – LM ={st | s is in L and t is in M}
- Kleene Closure – L* (zero or more concatenations of L)
- Positive Closure – L+ (one or more concatenations of L)

39. Write a regular expression for an identifier.

- An identifier is defined as a letter followed by zero or more letters or digits. The regular expression for an identifier is given as

  letter (letter | digit)*

40. How the regular expression can be defined in specification of the language.

1. ε is a regular expression that denotes {ε}, the set containing empty string.
2. If *a* is a symbol in Σ, then *a* is a regular expression that denotes {*a*}, the set containing the string *a*.
3. Suppose *r* and *s* are regular expressions denoting the language L(*r*) and L(*s*), then
   - (*r*) |(*s*) is a regular expression denoting L(*r*) ∪L(*s*).
   - (*r*)(*s*) is regular expression denoting L (*r*) L(*s*).
   - (*r*) * is a regular expression denoting (L (*r*))*.
   - (*r*) is a regular expression denoting L (*r*).

41. Mention the various notational short hands for representing regular expressions.

- One or more instances
- Zero or one instance
- Character classes
- Non regular sets

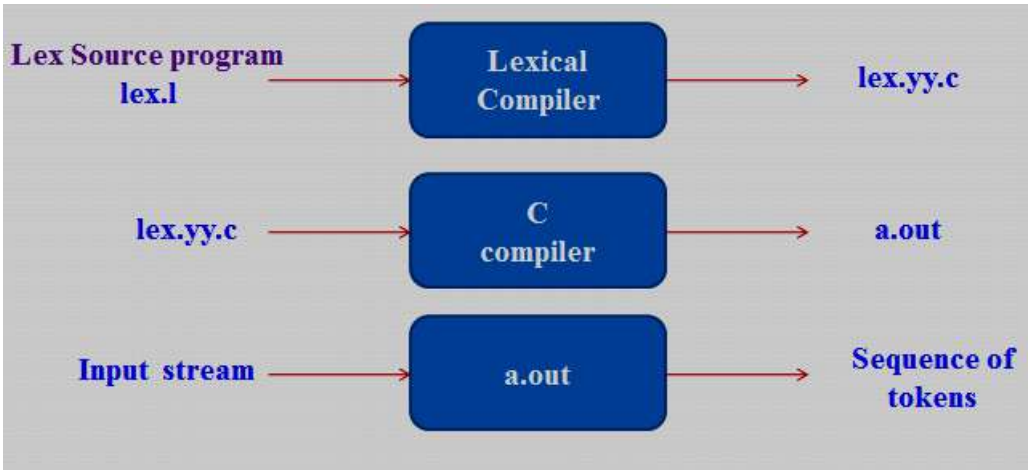
42. What is transition diagram? **(NOV 2012, 2016)**

- An intermediate step in the construction of a lexical analyzer, we first produce a stylized flowchart called a *transition diagram.*
- Transition diagram depicts the actions that take place when a lexical analyzer is called by the parser to get the next token.

43. Define Lex complier?

A particular tool called *Lex*, used to specify lexical analyzers for a variety of languages. We refer to the tool as the *Lex compiler* and to its input specification as the Lex language.

44. How to create a lexical analyzer with Lex?



45. List out the parts on Lex specifications. (OR) Give the structure of a Lex program with example.

(MAY 2012, 2015)

A Lex program consists of three parts:

declarations

%%

translation rules

%%

auxiliary procedures

- The declarations section includes declarations of variables, manifest constants, and regular definitions.
- The translation rules of a Lex program are statement of the form

  p_1 { $action_1$}

  p_2 { $action_2$}

  …… …..

  p_n { $action_n$}

- The auxiliary procedures are needed by the actions. These procedures can be compiled separately and loaded with the lexical analyzer.

46. Compare and contrast Compilers and Interpreters. **(MAY 2015, 2017)**

Compilers	Interpreters
Compiler Takes Entire program as input	Interpreter Takes Single instruction as input
Intermediate Object Code is Generated	No Intermediate Object Code is Generated
Memory Requirement **:** More (Since Object Code is Generated)	Memory Requirement **is** Less
Errors are displayed after entire program is checked	Errors are displayed for every instruction interpreted
Example **:** C Compiler	Example **:** BASIC

47. What is meant by tokens? **(NOV 2016, 2017)**

- Tokens - Sequence of characters that have a collective meaning.
- The tokens are keywords, operators, identifiers, constants, literal strings, and punctuation symbols such as parentheses, commas, and semicolons.

48. Write regular expression for the following language over the alphabet over the alphabet ∑ = {a, b}. All strings that contain an even number of b's. **(NOV 2017)**

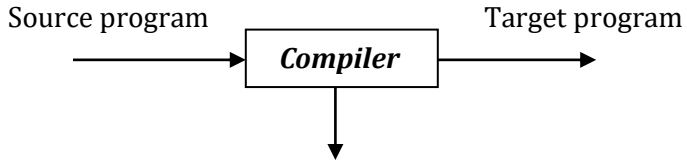
- The regular expression for the following language over the alphabet over the alphabet ∑ = {a, b}.
- All strings that contain an even number of b's.

a*(ba*ba*)*

11 MARKS

1. Write a short note on Compiler Design? (6 MARKS)

A ***compiler*** is a program that reads a program written in one language – the source language and translates it into an equivalent program in another language – the target language. As an important part of this translation process, the compiler reports to its user the presence of errors in the source program.



Error messages

- At first, the variety of compilers may appear overwhelming. There are thousands of source languages, ranging from traditional programming languages such as FORTRAN and Pascal to specialized languages in every area of computer application.
- Target languages are equally as varied; a target language may be another programming language, or the machine language of any computer between a microprocessor and a supercomputer.

Compilers are sometimes classified as:

- Single-pass
- Multi-pass
- Load-and-go
- Debugging or
- Optimizing

- Throughout the 1950's, compilers were considered notoriously difficult programs to write. The first FORTRAN compiler, for example, took 18 staff-years to implement.

Analysis-Synthesis Model of Compilation

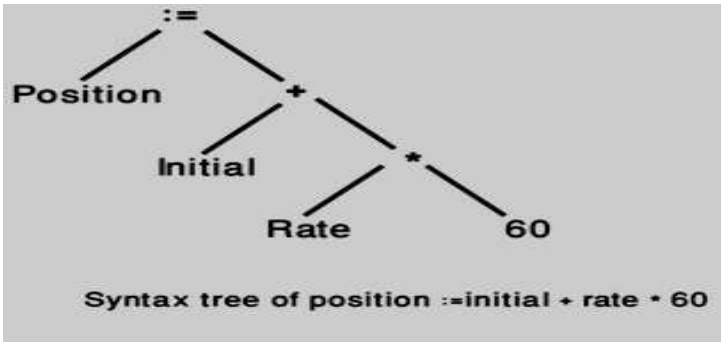
There are two parts to compilation as

1. Analysis part
2. Synthesis part

- The *analysis part* breaks up the source program into constituent pieces and creates an intermediate representation of the source program.
- The *synthesis part* constructs the desired target program from the intermediate representation.

- During analysis, the operations implied by the source program are determined and recorded in a hierarchical structure called a ***tree.***
- Often, a special kind of tree called a ***syntax tree*** is used, in syntax tree each node represents an operation and the children of the node represent the arguments of the operation.

For example, a syntax tree of an assignment statement is **position: = initial + rate * 60.**



Syntax tree of position :=initial + rate * 60

Many software tools that manipulate source programs first perform some kind of analysis. Some examples of such tools include:

1. Structure editors
2. Pretty printers
3. Static checkers
4. Interpreters

Structure editors

A *structure editor* takes as input a sequence of commands to build a source program. The structure editor not only performs the text-creation and modification functions of an ordinary text editor, but it also analyzes the program text, appropriate hierarchical structure on the source program. It is useful in the preparation of source program.

Pretty printers

A *pretty printer* analyzes a program and prints it in a way that the structure of the program becomes clearly visible. For example, comments may appear in a special font.

Static checkers

A static checker reads a program, analyzes it, and attempts to discover potential bugs without running the source program.

Interpreters

Interpreters are frequently used to execute command languages, since each operator executed in command languages is usually a complex routine such as an editor or compiler.

The analysis portion in each of the following examples is similar to that of a conventional compiler.

- Text formatters
- Silicon compilers
- Query interpreters

Text formatters

A *text formatter* takes input that is a stream of characters, most of which is text to be typeset, and it includes commands to indicate paragraphs, figures, or mathematical structures like subscripts and the superscripts.

Silicon compilers

A *silicon compiler* has a source language that is similar or identical to a conventional programming language. However, the variables of the language represent, not locations in memory, but also logical signals (0 or 1) or groups of signals in a switching circuit. The output is a circuit design in an appropriate language.

Query interpreters

A *Query interpreter* translates a predicate containing relational and Boolean operators into commands to search a database for records satisfying that predicate.

2. Explain the Analysis of the Source Program? (11 MARKS) (NOV 2015)

In compiling, analysis consists of three phases:

1. Linear Analysis
2. Hierarchical Analysis
3. Semantic Analysis

Linear Analysis

- *Linear analysis*, in which the stream of characters making up the source program is read from left-to-right and grouped into tokens that are sequences of characters having a collective meaning
- In a compiler, linear analysis is called lexical ***analysis or scanning.***

For example, in lexical analysis the characters in the assignment statement **position: = initial + rate * 60** would be grouped into the following tokens:

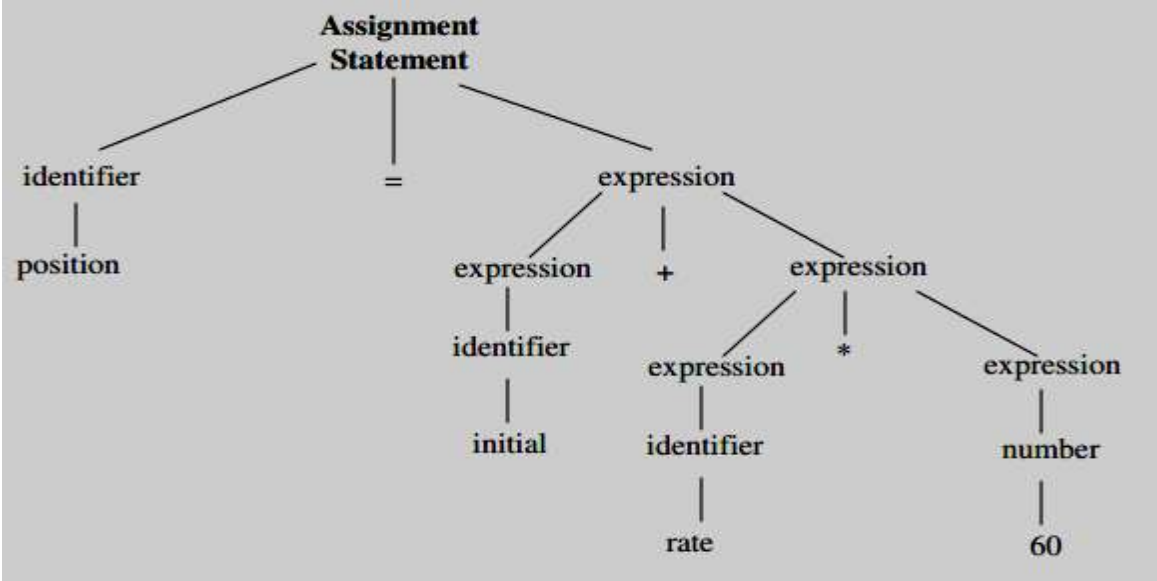
1. The identifier position.
2. The assignment symbol :=
3. The identifier initial.
4. The plus sign.
5. The identifier rate.
6. The multiplication sign.
7. The number 60.

- The blanks separating the characters of these tokens would normally be eliminated during the lexical analysis.

Hierarchical Analysis

- *Hierarchical analysis* is called ***parsing*** **or** ***syntax analysis.***

- Hierarchical analysis involves grouping the tokens of the source program into grammatical phases that are used by the compiler to synthesize output.
- The grammatical phrases of the source program are represented by a parse tree.



Parse tree for position: = initial + rate * 60

- In the expression **initial + rate * 60**, the phrase rate * 60 is a logical unit because the usual conventions of arithmetic expressions tell us that the multiplication is performed before addition.
- Because the expression **initial + rate** is followed by a *, it is not grouped into a single phrase by itself.

- The hierarchical structure of a program is usually expressed by *recursive rules*. For example, the following rules, as part of the definition of expression:

  1. Any *identifier* is an expression.
  2. Any *number* is an expression
  3. If $expression_1$ and $expression_2$ are expressions, then so are
     - $expression_1 + expression_2$
     - $expression_1 * expression_2$
     - $(expression_1)$

- Rules (1) and (2) are non-recursive basic rules, while (3) defines expressions in terms of operators applied to other expressions.

Similarly, many languages define statements recursively by rules such as:

1. If $identifier_1$ is an identifier and $expression_2$ is an expression, then

   $identifier_1 := expression_2$

   is a statement.

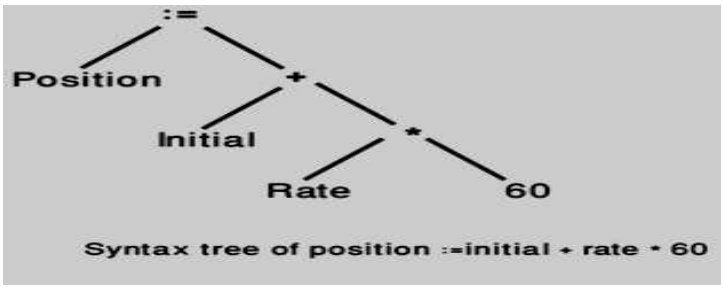
2. If *expression*$_1$ is an expression and *statement*$_2$ is a statement, then

while *(expression*$_1$*)* ***do*** *statement*$_2$

if *(expression*$_1$*)* ***then*** *statement*$_2$

are statements.

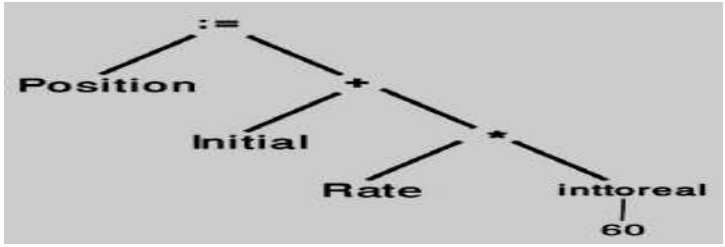
A ***syntax tree*** is a compressed representation of the parse tree in which the operators appear as the interior nodes and the operands of an operator are the children of the node for that operator.



Syntax tree of position :=initial + rate * 60

Semantic Analysis

- The *semantic analysis* phase checks the source program for semantic errors and gathers type information for the subsequent code-generation phase.
- It uses the hierarchical structure determined by the syntax-analysis phase to identify the operators and operand of expressions and statements.
- An important component of semantic analysis is *type checking.*
- The compiler checks that each operator has operands that are permitted by the source language specification.
- For example, when a binary arithmetic operator is applied to an integer and real.
- In this case, the compiler may need to be converting the integer to a real. As shown in figure given below.



3. Explain the various Phases of a Compiler with an example? (11 MARKS) (NOV 2011, 2012, 2013, 2016, 2017) (MAY 2012, 2013, 2014, 2015, 2016, 2018)

A compiler operates in phases, each of which transforms the source program from one representation to another.

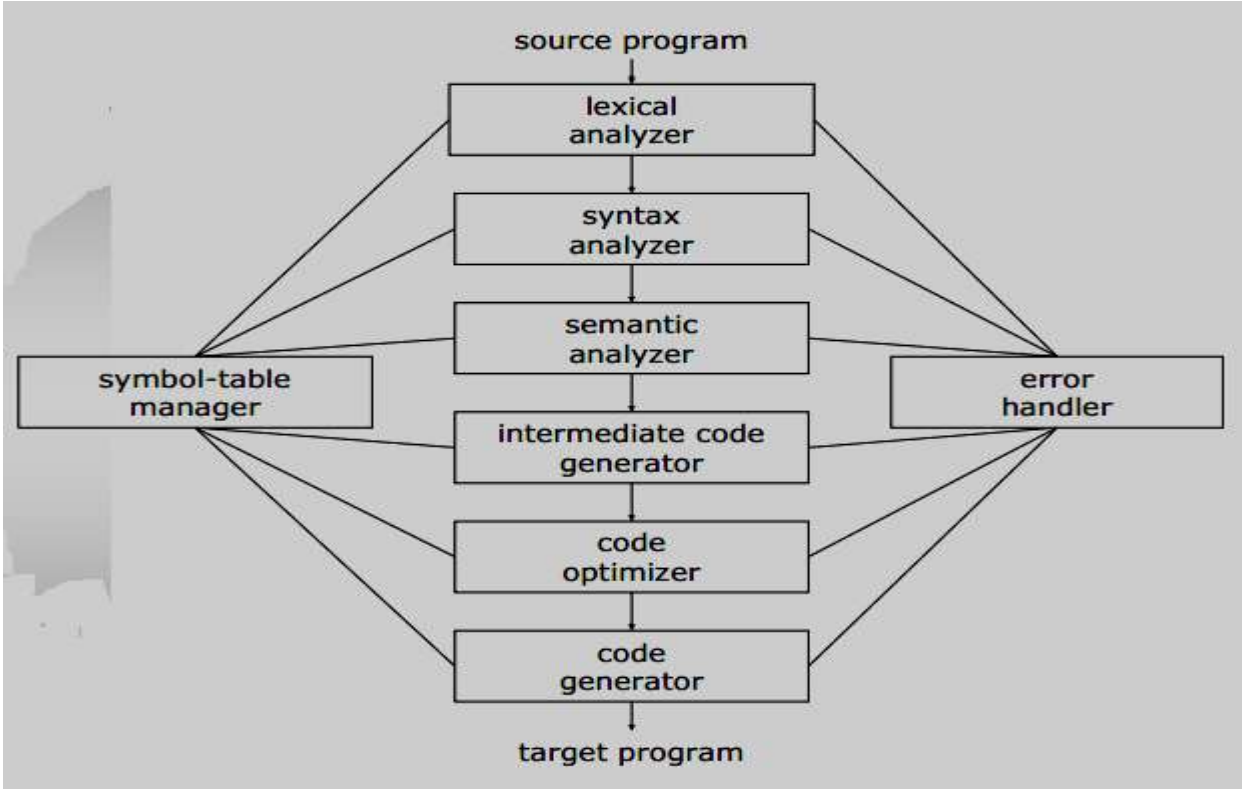
The six phases of a complier are

1. Lexical Analyzer
2. Syntax Analyzer
3. Semantic Analyzer
4. Intermediate code generator
5. Code optimizer
6. Code generator

Two other activities are

- Symbol table Manager
- Error handler

A typical decomposition of a compiler is shown in fig given below



Phases of a compiler

<u>The Analysis Phases</u>

As translation progresses, the compiler's internal representation of the source program changes. Consider the translation of the statement,

position: = initial + rate * 10

Lexical analyzer

- The ***lexical analysis*** phase reads the characters in the source program and groups them into a stream of tokens in which each token represents a logically cohesive sequence of characters, such as an identifier, a keyword (if, while, etc.), a punctuation character or a multi-character operator like :=.
- In a compiler, *linear analysis* is called ***lexical analysis* or *scanning.***
- ***Linear analysis*** in which the stream of characters making up the source program is read from left-to-right and grouped into tokens that are sequences of characters.
- The character sequence forming a token is called the ***lexeme*** for the token.
- Certain tokens will be augmented by a '*lexical value*'. For example, when an identifier like *rate* is found, the lexical analyzer not only generates a token id, but also enters the lexeme rate into the symbol table, if it is not already there.

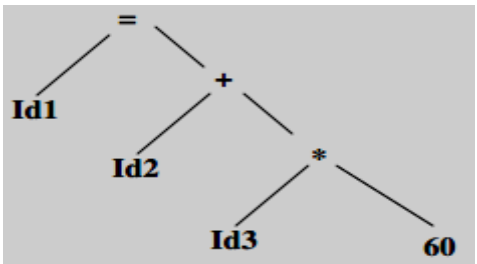
Consider $\mathbf{id_1}$, $\mathbf{id_2}$ and $\mathbf{id_3}$ for **position, initial, and rate** respectively, that the internal representation of an identifier is different from the character sequence forming the identifier.

The representation of the statement given above after the lexical analysis would be:

$$\mathbf{id_1 := id_2 + id_3 * 10}$$

Syntax analyzer

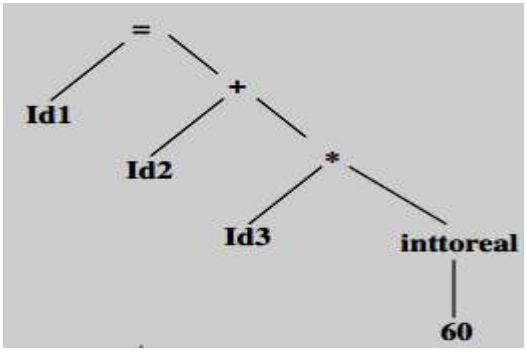
- ***Hierarchical analysis*** involves grouping the tokens of the source program into grammatical phases that are used by the compiler to synthesize output.
- *Hierarchical analysis* is called ***parsing* or *syntax analysis.***
- Syntax analysis imposes a hierarchical structure on the token stream, which is shown by syntax trees.



Semantic analyzer

- The ***Semantic analysis,*** it checks the source program for semantic errors and gathers type information for the subsequent code generation phase.

- It uses the hierarchical structure determined by the syntax analysis phase to identify the operators and operands of expressions and statements.
- Compiler report an error, if **real number** is used to **index** an array.
- The bit pattern representing an integer is generally different from the bit pattern for a real, even they have the same value.
- For example, the identifiers position, initial, rate declared to be **real** and that 60 by itself assumed to be **integer.**
- The general approach is to convert the integer to a real.



Intermediate code generator

- After Syntax and semantic analysis, some compilers generate an explicit intermediate representation of the source program. The intermediate representation as a program for an abstract machine.
- This intermediate representation should have two important properties;
  - it should be easy to produce, and
  - it should be easy to translate into the target program
- An intermediate form called "*three-address code*, "which is like the assembly language for a machine in which every memory location can act like a register.
- Three-address code consists of a sequence of instructions, each of which has at most three operands.
- The source program might appear in three-address code as

temp1: = inttoreal (60)
temp2: = id3 * temp1
temp3: = id2 + temp2
id1: = temp3

This intermediate form has several properties:

1. First, each three address instruction has at most one operator in addition to the assignment. Thus, when generating these instructions, the compiler has to decide on the order in which operations are to be done; the multiplication precedes the addition in the source program.
2. Second, the compiler must generate a temporary name to hold the value computed by each instruction.
3. Third, some "three-address" instructions have fewer than three operands.

Code Optimization

- The *code optimization phase* attempts to improve the intermediate code, so that faster-running machine code will result.
- For example, a natural algorithm generates the intermediate code, using an instruction for each operator in the tree representation after semantic analysis, even though there is a better way to perform the same calculation, using the two instructions.

  temp1:= id3 * 60.0

  id1 := id2 + temp1

- There is nothing wrong with this simple algorithm, since the problem can be fixed during the code-optimization phase.
- That is, the compiler can deduce that the conversion of 60 from integer to real representation can be done once and for all at compile time, so the inttoreal operation can be eliminated.
- There is a great variation in the amount of code optimization different compilers.
- 'Optimizing compilers', a significant fraction of the time of the compiler is spent on this phase.

Code Generation

- The final phase of the compiler is the *generation of target code*, consisting normally of relocatable machine code or assembly code.
- Memory locations are selected for each of the variables used by the program. Then, intermediate instructions are each translated into a sequence of machine instructions that perform the same task.
- The assignment of variables to registers.

For example, using registers 1 and 2, the translation of the code as

```
MOVF   id3,   R2
MULF   #60.0, R2
MOVF   id2,   R1
ADDF   R2,    R1
MOVF   R1,    id1
```

- The first and second operands of each instruction specify a source and destination, respectively.
- The ***F*** in each instruction tells us that instructions deal with floating-point numbers.
- This code moves the contents of the address id3 into register 2, and then multiplies it with the real-constant 60.0.
- The **#** signifies that 60.0 is to be treated as a constant.
- The third instruction moves id2 into register 1 and adds to it the value previously computed in register 2.
- Finally, the value in register 1 is moved into the address of id1.

Symbol Table Management

- An essential function of a compiler is to record the identifiers used in the source program and collect information about various attributes of each identifier.
- These attributes may provide information about the storage allocated for an identifier, its type, its scope and in case of procedure names, such things at the number and types of its arguments and methods of passing each argument and type returned.
- The ***symbol table*** is a data structure containing a record for each identifier with fields for the attributes of the identifier.
- The data structure allows us to find the record for each identifier quickly and to store or retrieve data from that record quickly.
- Whenever an identifier is detected by a lexical analyzer, it is entered into the symbol table. The attributes of an identifier cannot be determined by the lexical analyzer.
- However, the attributes of an identifier cannot normally be determined during lexical analysis.

  For example, in a ***Pascal declaration*** like

  var position, initial, rate : real;

- The type real is not known when position, initial and rate are seen by the lexical analyzer.
- The remaining phases get information about identifiers into the symbol table and then use this information in various ways.

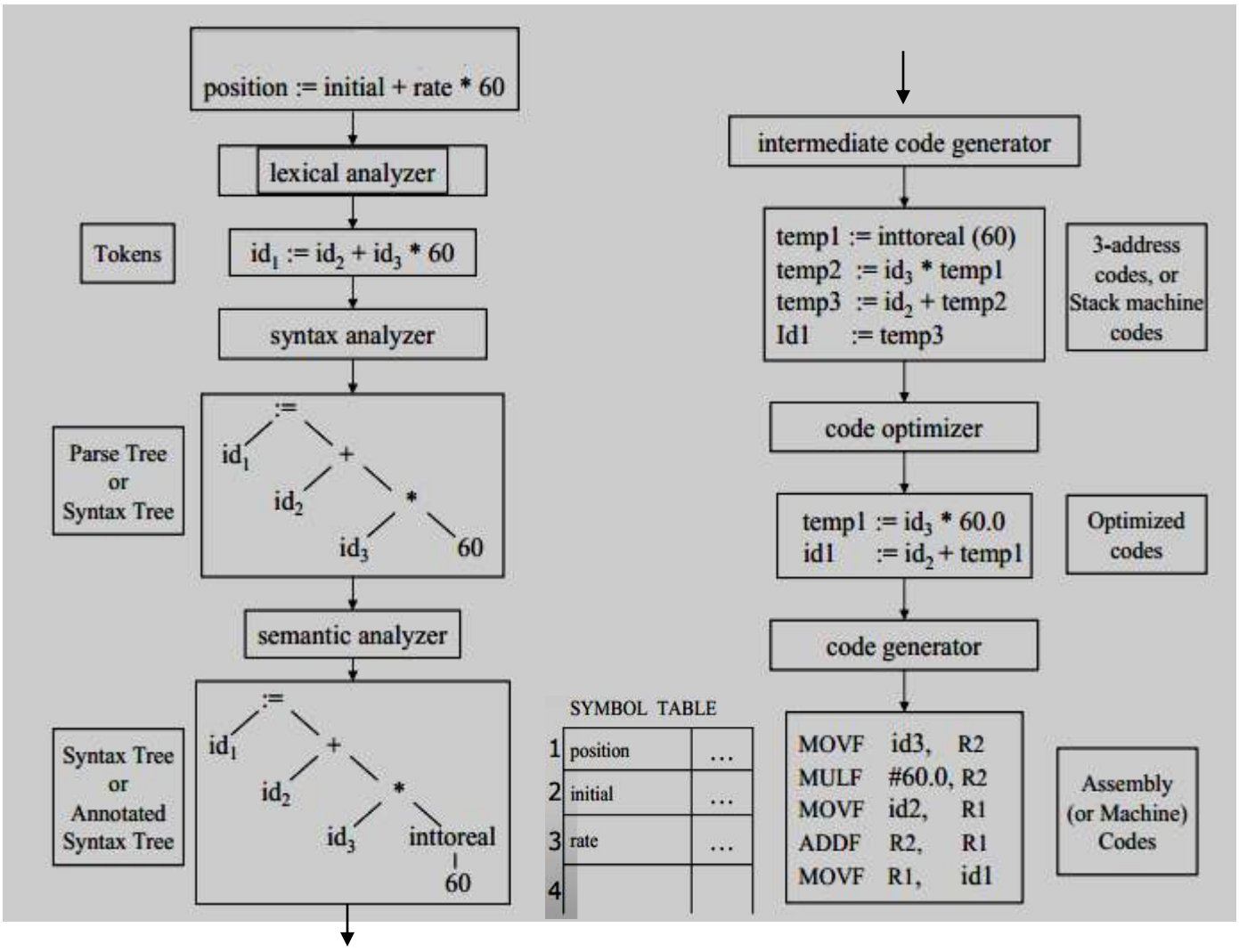
Error Detection and Reporting

- Each phase can encounter errors. However, after detecting an error, a phase must somehow deal with that error, so that compilation can proceed, allowing further errors in the source program to be detected.
- A compiler that stops when it finds the first error is not as helpful as it could be.
- The *syntax and semantic analysis* phases usually handle a large fraction of the errors detectable by the compiler.
- The *lexical phase* can detect errors where the characters remaining in the input do not form any token of the language.

- Errors where the token stream violates the structure rules (syntax) of the language are determined by the syntax analysis phase.
- During semantic analysis the compiler tries to detect the right syntactic structure but no meaning to the operation involved.
- Example: we try to add two identifiers, one of which is the name of an array and the other the name of the procedure.

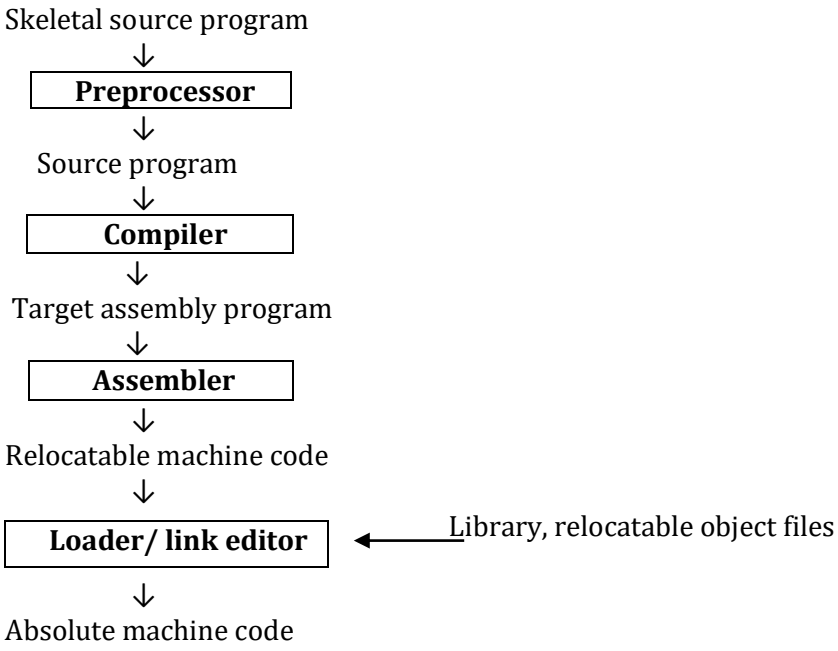
Example

Consider the translation of the statement, **position: = initial + rate * 10**



4. Explain in detail about Cousins of the Compiler? (11 MARKS) (MAY 2012) (NOV 2017)

The input to a compiler may be produced by one or more preprocessors, and further processing of the compiler's output may be needed before running machine code is obtained.



Preprocessors

Preprocessors produce input to compilers. They may perform the following functions:

- Macro Processing
- File inclusion
- "Rational" Preprocessors
- Language extensions

Macro Processing

A preprocessor may allow a user to define macros that are shorthand's for longer constructs.

File inclusion

- A preprocessor may include header files into the program text.
- *For example,* the C preprocessor causes the contents of the file ***<global.h>*** to replace the statement ***#include <global.h>*** when it processes a file containing this statement.

"Rational" Preprocessors

- These processors augment older languages with more modern flow-of-control and data-structuring facilities.
- *For example,* such a preprocessor might provide the user with built-in macros for constructs like while statements or if statements.

Language extensions

- These processors attempt to add capabilities to the language by what amounts to built-in macros.
- *For example,* the language Equal is a database query language embedded in C. Statements beginning with ## are taken by the preprocessor to be database-access statements, unrelated to C, and are translated into procedure calls on routines that perform the database access.

Macro processors deal with two kinds of statements:

1. Macro definition and
2. Macro use

- Definitions are normally indicated by unique character or keyword like define or macro. They consist of a name for the macro being defined and a body, forming its definition.
- The use of macro consists of naming the macro and supply actual parameters, (i.e.) values for its formal parameters.

Assemblers

- Some compilers produce assembly code that is passed to an assembler for further processing.
- Other compilers perform the job of the assembler, producing relocatable machine code that can be passed directly to the loader/link-editor.
- *Assembly code* is a mnemonic version of machine code, in which names are used instead of binary codes for operations, and names are also given to memory addresses.
- A typical sequence ***b := a + 2,*** the assembly instructions might be

```
MOV a, R1
ADD #2, R1
MOV R1, b
```

- This code moves the contents of the address **a** into register 1, then adds the constant **2** to it, treating the contents of register 1 as a fixed-point number, and finally stores the result in the location named by **b**. thus, it computes b:=a+2.

Two - Pass Assembly

The simplest form of assembler makes two passes over the input, where a *pass* consists of reading an input file once.

- In the *first pass*, all the identifiers that denote storage locations are found and stored in a symbol table. Identifiers are assigned storage locations as they are encountered for the first time.

Identifiers	Address
a	0
b	4

- In the *second pass*, the assembler scans the input again. This time, it translates each operation code into the sequence of bits representing that operation in machine language and it translates each identifier representing a location into address given for that identifier in the symbol table.
- The output of the second pass is usually *relocatable* machine code, that it can be loaded starting at any location L in memory.

Loaders and Link-Editors

- Usually, a program called a *loader* performs the two functions of *loading* and *link-editing.*
- The process of *loading* consists of taking relocatable machine code, altering the relocatable addresses, and placing the altered instructions and data in memory at the proper location.
- The *link-editor* allows us to make a single program from several files of relocatable machine code.
- These files may be the result of several different compilation and one or more library files of routines provided by the system.
- If the files are to be used together, there may be external references, in which the code of one file refers to a location in another file.

5. Write a short note on Grouping of Phases? (5 MARKS)

In an implementation, activities from more than one phase are often grouped together.

Front and Back Ends

- The phases are collected into a *front end* and a *back end.*
- The ***front end*** consists of those phases that depend primarily on the source language and are largely independent of the target machine.

  These normally include
  - lexical analysis
  - syntactic analysis
  - semantic analysis
  - the creating of the symbol table
  - the generation of intermediate code.
  - code optimization can be done by the front end as well.

- The front end also includes the error handling that goes along with each of these phases.
- The ***back end*** of compiler includes those portions that depend on the target machine and generally those portions do not depend on the source language, just the intermediate language.

These normally include

- Code optimization
- Code generation
- Error handling and
- Symbol-table operations

Passes

- Several phase of compilation are usually implemented in a single ***pass*** consisting of reading an input file and writing an output file.
- For several phases to be grouped into one pass and for the activity of these phases to be interleaved during the pass.
- For example, lexical analysis, syntax analysis, semantic analysis and intermediate code generation might be grouped into one pass.
- The token stream after lexical analysis may be translated directly into intermediate code.

Reducing the Number of Passes

- It is desirable to have relatively few passes, since it takes time to read and write intermediate files. If we group several phases into one pass, it may be forced to keep the entire program in memory, because one phase may need information in a different order then a previous phase produces it.
- The grouping into one pass presents few problems.
- The interface between lexical and syntax analyzers can often be limited to a single token.
- It is very hard to perform code generation until the intermediate representation has been completely generated.

6. State the different Compiler Construction Tools and their use. (6 MARKS) (MAY 2013, 2017) (NOV 2015, 2017)

- The compiler writer, like any programmer, can profitably use tools such as debuggers, version managers, profilers and so on.
- In addition to these software-development tools, other more specialized tools have been developed for helping implement various phases of a compiler.
- The first compilers were written; systems to help with the compiler-writing process appeared.

These systems have often been referred to as

- **Compiler-compilers,**
- **Compiler-generators, or**
- **Translator-writing systems**

- The general tools have been created for the automatic design of specific compiler components.
- These tools use specialized languages for specifying and implementing the component, and many use algorithms that are quite sophisticated.
- The most successful tools are those that hide the details of the generation algorithm and produce components that can be easily integrated into the remainder of a compiler.

The various compiler construction tools are

1. Parser generators
2. Scanner generators
3. Syntax-directed translation engines
4. Automatic code generators
5. Data-flow engines

Parser generators

- These produce syntax analyzers, normally from input that is based on a context-free grammar.
- In early compilers, syntax analysis consumed not only a large fraction of the running time of a compiler, but a large fraction of the intellectual effort of writing a compiler.
- This phase is considered one of the easiest to implement.

Scanner generators

- These tools automatically generate lexical analyzers, normally from a specification based on regular expressions.
- The basic organization of the resulting lexical analyzer is in effect a finite automaton.

Syntax directed translation engines

- These produce collections of routines that walk the parse tree, generating intermediate code.
- The basic idea is that one or more "translations" are associated with each node of the parse tree, and each translation is defined in terms of translations at its neighbor nodes in the tree.

Automatic code generators

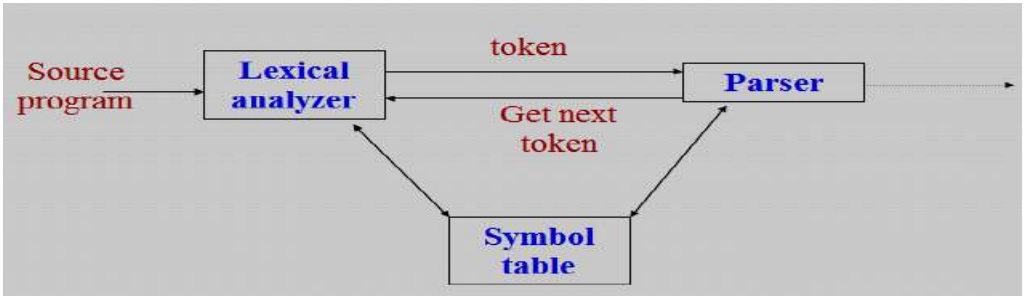
- Such a tool takes a collection of rules that define the translation of each operation of the intermediate language into the machine language for the target machine.
- The rules must handle the different possible access methods for data.
- Eg; variables may be in registers, in a fixed location in memory or may be allocated a position on a stack. The basic technique is *"**template matching".***

Data-flow engines

- Much of the information needed to perform good code optimization involves "data-flow analysis," the gathering of information how values are transmitted from one part of a program to each other part.

7. Discuss the Role of the Lexical Analyzer? (11 MARKS) (NOV 2012, 2014) (MAY 2014, 2016, 2017, 2018)

- The *lexical analyzer* is the first phase of a compiler.
- Its main task is to read the input characters and produce as output a sequence of tokens that the parser uses the syntax analysis.
- This is implemented by making the lexical analyzer be a sub-routine or a co-routine of the parser.
- Upon receiving a "get next token" command from the parser, the lexical analyzer reads input characters until it can identify the next token.



Interaction of lexical analyzer with parser

The lexical analyzers is the part of the compiler that reads the source text, it may also perform certain secondary tasks at the user interface.

- One task is stripping out from the source program *comments and white space* in the form of *blank, tab and new line characters.*
- Another is *error messages* from the compiler in the source program.

The lexical analyzers are divided into a cascade two phases are

1. *Scanning* → is responsible for doing simple tasks.
2. *Lexical analysis* →more complex operations

For example, a FORTRAN compiler might use a scanner to eliminate blanks from the input.

Issues in Lexical Analysis

There are several reasons for separating the analysis phase of compiling into lexical analysis and parsing.

i. Simpler design is the most important consideration.
   - Comments and white space have already been removed by lexical analyzer.

ii. Compiler Efficiency is improved.
   - A large amount of time is spent reading the source program and partitioning it into tokens.
   - Specialized buffering techniques for reading input characters and processing tokens can significantly speed up the performance of a compiler.

iii. Compiler Portability is enhanced.

- Input alphabet peculiarities and other device-specific anomalies can be restricted to the lexical analyzer.
- The representation of a special or non-standard symbol such as ↑ in Pascal can be isolated in the lexical analyzer.

Tokens, Patterns, and Lexemes

- *Tokens*- Sequence of characters that have a collective meaning.
- *Patterns*- There is a set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a pattern associated with the token.
- *Lexeme*- A sequence of characters in the source program that is matched by the pattern for a token.

Token	lexeme	patterns
const	const	const
if	if	if
relation	<, <=, =, < >, >, >=	< or <= or = or < > or >= or >
id	pi, count, D2	letter followed by letters and digits
num	3.1416, 0, 6.02E23	any numeric constant
literal	"core dumped"	any characters between " and " except "

Attributes for Tokens

- When more than one pattern matches a lexeme, the lexical analyzer must provide information about the particular lexeme that matched to the phases of a compiler.
- For example, the pattern **num** matches both the strings 0 and 1.
- The lexical analyzer collects information about tokens into their associated attributes.
- The *tokens* influence *parsing decisions*; the *attributes* influence the *translation of tokens.*
- A token has usually only a single attribute – a pointer to the symbol-table entry in which the information about the token is kept; the pointer becomes the attribute for the token.

The tokens and associated attribute-values for the FORTRAN statement

E = M * C ** 2

<id, pointer to symbol-table entry for R>

<assign_op, >

<id, pointer to symbol-table entry for M>

<mult_op, >

<id, pointer to symbol-table entry for C>

<exp_op, >

<num, integer value 2>

Lexical Errors

Possible error recovery actions or Panic mode recovery are

1. Deleting an extraneous character
2. Inserting a missing character
3. Replacing an incorrect character by a correct character
4. Transposing two adjacent characters

8. Explain the Input Buffering with sentinels. (6 MARKS) (NOV 2013)

- The two- buffer input scheme is useful when look-ahead on the input is necessary to identify tokens.
- The techniques for speeding up the lexical analyzer, use the "sentinels "to mark the buffer end.

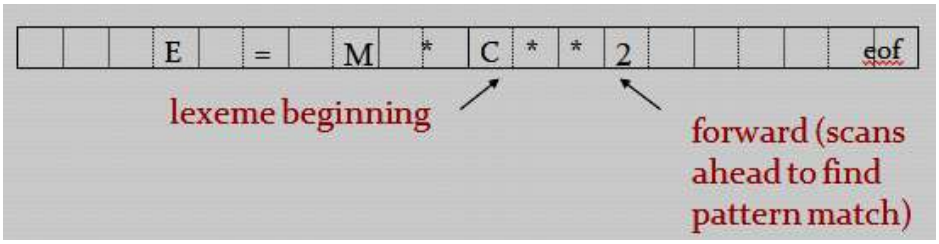
The three general approaches to the implementation of a lexical analyzer

1. Use a lexical analyzer generator such as the Lex compiler to produce a regular expression based specification. In this case, the generator provides for reading and buffering the input.
2. Write the lexical analyzer in a conventional systems programming languages using the I/O facilities of that language to read the input.
3. Write the lexical analyzer in assembly language and reading of input.

The lexical analyzer is the only phase of the compiler that reads the source program character-by-character; it is possible to spend a considerable amount of time in the lexical analysis phase.

Buffer Pairs

- Two pointers to the input buffer are maintained.
- The string of characters between the pointers is the current lexeme.
- Initially, both pointers point to the first character of the next lexeme to be found.
- Forward pointer, scans ahead until a match for a pattern is found.
- Once the next lexeme is determined, the forward pointer is set to the character at its right end.
- After the lexeme is processed, both pointers are set to the character immediately past the lexeme.
- The comments and white space can be treated as patterns that yield no token.
- A buffer into two N-character halves, where N is the no.of characters on one disk block, eg. 1024 or 4096.



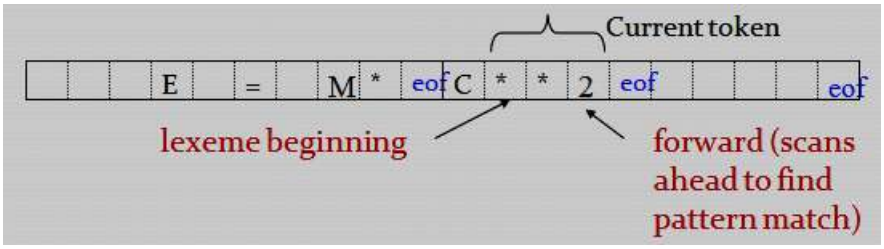
- If the forward pointer is to move past the halfway mark, the right half is filled with N new input characters.
- If the forward pointer is to move past the right end of the buffer, the left half is filled with N new characters and the forward pointer wraps around to the beginning of the buffer.

Code to advance forward pointer

```
if forward at the end of first half then begin
        reload second half ;
        forward := forward + 1;
end
else if forward at end of second half then begin
        reload first half ;
        move forward to beginning of first half
end
else forward := forward + 1;
```

Sentinels

- The *sentinel* is a special character that cannot be part of the source program.
- Each buffer half to hold a sentinel character at the end (eof).



Lookahead code with sentinels

```
forward := forward + 1 ;
if forward↑ = eof then begin
    if forward at end of first half then begin
        reload second half ;
        forward := forward + 1
end
else if forward at end of second half then begin
        reload first half ;
        move forward to beginning of first half
end
else / * eof within buffer signifying end of input * /
        terminate lexical analysis
end
```

9. Explain in detail about the Specification of Tokens? (11 MARKS)

Regular expressions are an important notation for specifying lexeme patterns. Each pattern matches a set of strings, so regular expressions will serve as names for set of strings.

(i) Strings and Languages

- The term *alphabet or character class* denotes any finite set of symbols. Typical examples of symbols are letters and characters.
- The set {0, 1} is the *binary alphabet*.
- ASCII and EBCDIC are two examples of computer alphabet.
- A ***string*** over some alphabet is a finite sequence of symbols drawn from that alphabet.
- The length of string s, usually written |s|, is the number of occurrences of symbols in s.
- The *empty string* denoted Ɛ, is a special string of length zero.
- The term ***language*** denotes any set of strings over some fixed alphabet. Abstract languages like Φ, the empty set, or {ε},the set containing only the empty string, are languages.
- If x and y are strings, then the *concatenation* of x and y is also string, denoted xy, is the string formed by appending y to x.

  For example, if x = ban and y = ana, then xy = banana.
- The empty string Ɛ is the identity element under concatenation; that is, for any string s, SƐ = ƐS= s.

(ii) Operations on Languages

There are several important operations that can be applied to languages.

In lexical analysis

- Union
- Concatenation
- Closure

OPERATION	DEFINITION
union of L and M written $L \cup M$	$L \cup M = \{s \mid s \text{ is in } L \text{ or } s \text{ is in } M\}$
concatenation of L and M written LM	$LM = \{st \mid s \text{ is in } L \text{ and } t \text{ is in } M\}$
Kleene closure of L written L*	$L^* = \bigcup_{i=0}^{\infty} L^i$ L* denotes "zero or more concatenations of " L
positive closure of L written L^+	$L^+ = \bigcup_{i=1}^{\infty} L^i$ L^+ denotes "one or more concatenations of " L

Example

- Let L be the set of letters {A, B, . . . , Z, a, b, . . . , z } and D be the set of digits {0,1,.. .9}.
- L and D are, respectively, the alphabets of uppercase and lowercase letters and of digits.

1. L U D is the set of letters and digits
2. LD is the set of strings consisting of a letter followed by digit
3. L^4 is the set of all 4-letter strings.
4. L^* is the set of all strings of letters, including e, the empty string.
5. L $(L \cup D)^*$ is the set of all strings of letters and digits beginning with a letter.
6. D^+ is the set of all strings of one or more digits.

(iii) Regular Expressions

- Regular expression is a notation for describing string. In Pascal, an identifier is a letter followed by zero or more letter or digits.
- The Pascal identifier as

letter (letter | digit) *

The rules is the specification of language denoted by

1. **ε** is a regular expression that denotes {ε}, the set containing empty string.
2. If ***a*** is a symbol in Σ, then *a* is a regular expression that denotes {*a*}, the set containing the string *a*.
3. Suppose ***r*** and ***s*** are regular expressions denoting the language L(*r*) and L(*s*), then
   - **a)** (*r*) |(*s*) is a regular expression denoting L(*r*) ∪ L(*s*).
   - **b)** (*r*)(*s*) is regular expression denoting L (*r*) L(*s*).
   - **c)** (*r*) * is a regular expression denoting (L (*r*))*.
   - **d)** (*r*) is a regular expression denoting L (*r*).

A language denoted by a regular expression is said to be a *regular set.*

Unnecessary parentheses can be avoided in regular expression

1. The unary operator * has the highest precedence and is left associative.
2. Concatenation has the second highest precedence and is left associative.
3. | has the lowest precedence and is left associative.

(iv) Regular Definitions

- For notation, give names to regular expressions and to define regular expressions using these names as if they were symbols.
- If Σ is an alphabet of basic symbols, then a regular definition is a sequence of definitions of the form:

$d_1 \rightarrow r_1$

$d_2 \rightarrow r_2$

…

$d_n \rightarrow r_n$

where each d_i is a distinct name, and each r_i is a regular expression.

Example

1. The set of *Pascal identifier* is the set of strings of letters and digits beginning with a letter.

   The regular definition is

   letter → A | B | C | … | Z | a | b | … | z

   digit → 0 | 1 | 2 | … | 9

   id → **letter (letter | digit)***

2. *Unsigned numbers* in Pascal are strings such as 5280, 39.37, 6.336E4 or 1.894E-4.

   The regular definition is

   digit → 0 | 1 | 2 | … | 9

   digits → **digit digit***

   optional_fraction → **. digits** | ∈

   optional_exponent → (**E** (+ | -| ∈) **digits**) | ∈

   num → **digits optional_fraction optional_exponent**

(v) Notational Shorthands

1. **One or more instances**

   Unary postfix operator + means "one or more instances of".

2. **Zero or one instance**

   Unary postfix operator ? means " zero or one instances of ".The regular definition for **num**

   digit → 0 | 1 | 2 | … | 9

   digits → **digit**$^+$

   optional_fraction → (. **digits**) ?

   optional_exponent → (E (+ | -) ? **digits)** ?

   num → **digits optional_fraction optional_exponent**

3. **Character classes**

- The notation [abc] where a, b and c are alphabet symbols denotes the regular expression a | b | c.
- The character class such as [a-z] denotes the regular expression a | b | … | z.
- Using character classes, we describe identifiers as being strings generated by the regular expression,

  [A – Z a – z] [A – Z a – z 0 – 9] *

10. Illustrate the steps involved in the Recognition of Tokens? (11 MARKS) (NOV 2011, 2015) (MAY 2013, 2018)

We considered the problem of how to specify tokens and recognize them.

Consider the following grammar

stmt → **if** *expr* **then** *stmt*

| **if** *expr* **then** *stmt* **else** *stmt*

| ∈

expr → *term* **relop** *term*

| *term*

term → *id*

| *num*

where the terminals **if, then**, **else**, **relop**, **id**, and **num** generate set of strings given by the following regular definitions:

if → if

then → then

else → else

relop → < | <= | > | >= | = | < >

id → **letter (letter | digit)***

num → **digit + (. digit +) ? (E(+ | -) ? digit +) ?**

The lexical analyzer will recognize the **keywords** *if, then, else*, as well as the lexemes denoted by ***relop, id***, and ***num.***

We assume lexemes are separated by **white space** consisting of *blanks, tabs, and newlines*. In lexical analyzer will strip out white space.

delim → **blank | tab | newline**

ws → **delim +**

If a match for ***ws*** is found, the lexical analyzer does not return a token to the parser.

- To construct a lexical analyzer that will isolate the lexeme for the next token in the input buffer and produce as output a pair consisting of the appropriate token and attribute value, using the translation table.
- The attribute values for the relational operators are given by the symbolic constants LT, LE, EQ, NE, GT,GE.

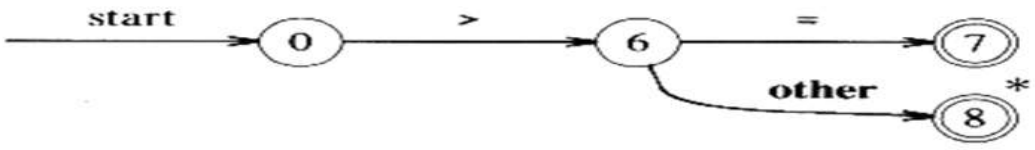
Regular Expression	Token	Attribute-Value
ws	-	-
if	if	-
then	then	-
else	else	-
id	id	pointer to table entry
num	num	pointer to table entry
<	relop	LT
<=	relop	LE
=	relop	EQ
< >	relop	NE
>	relop	GT
>=	relop	GE

Regular expression pattern for tokens

Transition diagram

- An intermediate step in the construction of a lexical analyzer, produce a stylized flowchart called ***a transition diagram.***
- Transition diagram depict the actions that take place when a lexical analyzer is called by the parser to get the next token.
- Transition diagram to keep track of information about characters that are seen as the forward pointer scans the input.
- Moving from position to position in the diagram as characters are read.
- Positions in a transition diagram are drawn as circles and are called **states.**
- The states are connected by arrow, called **edges**.
- Edges leaving state **s** have **labels** indicating the input characters that can next appear after the transition diagram has reached state ***s***.
- The **label other** refers to any character that is not indicated by any of the other edges leaving s.
- Transition diagram are **deterministic**, ie., no symbol can match the labels of two edges leaving one state.
- One state is labeled as the ***start state***; it is the initial state of the transition diagram where control resides when we begin to recognize a token.
- Certain states may have actions that are executed when the flow of control reaches that state.
- On entering a state we read the next input character.
- If there is an edge from the current state whose label matches this input character, we then go to the state pointed to by the edge.
- Otherwise we indicate failure.

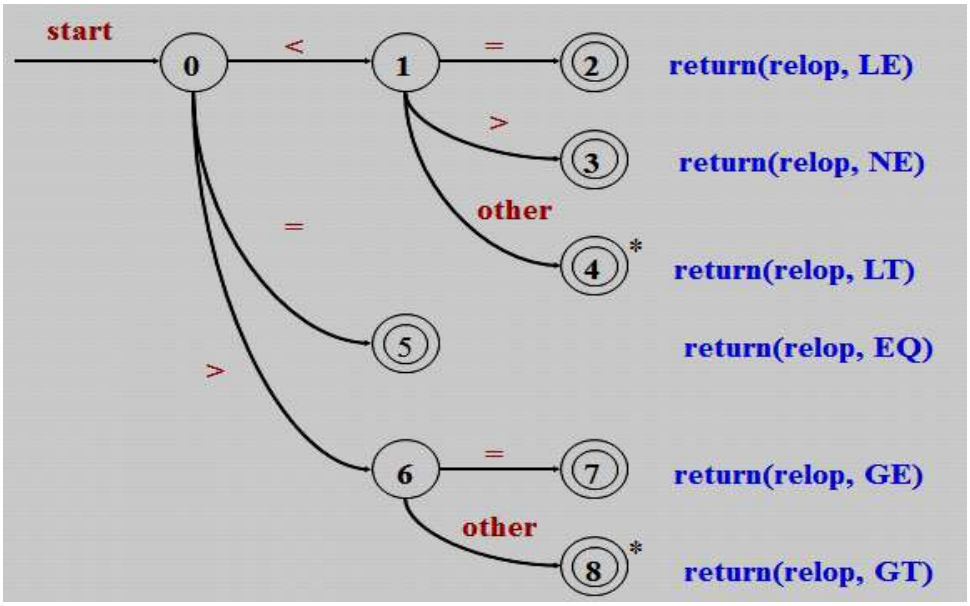
Transition Diagrams for >=



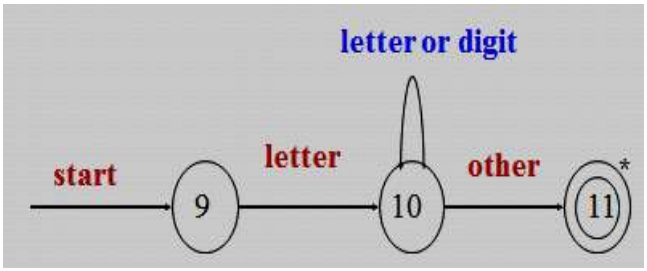
Transition diagram for >=

- start state : stare 0 in the above example
- If input character is >, go to state 6.
- other refers to any character that is not indicated by any of the other edges leaving s.

Transition diagram for relational operators

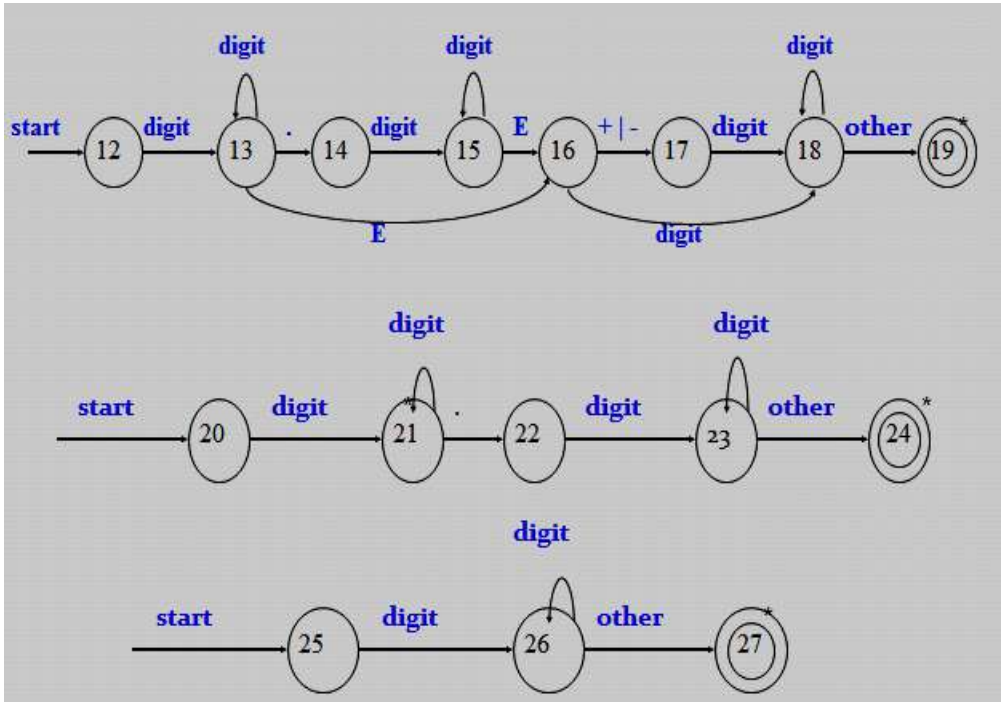


Transition diagram for identifiers and keywords

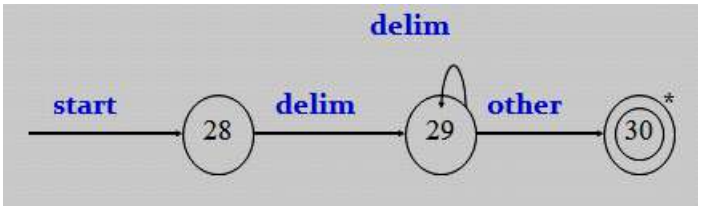


- gettoken( ): return token (**id, if, then**,…) if it looks the symbol table
- install_id( ): return 0 if keyword or a pointer to the symbol table entry if **id**

Transition diagram for digits (Unsigned Numbers)



Transition diagram for delim



Implementing a Transition diagram

- A sequence of transition diagram can be converted into a program to look for the tokens by the diagrams.
- The systematic approach that works for all transition diagram and constructs programs whose size is proportional to the number of states and edges in the diagrams.
- Each state gets a segment of code. If there are edges leaving a state, then its code reads a character and selects an edge to follow, if possible.
- A function **nextchar()** is used to read next character from the input buffer, advance the forward pointer at each call, and return the character read. If there is an edge labeled by the character read, or labeled by a character class containing the character read, then control is transferred to the code for the state pointed to by that edge.
- If there is no such edge, and the current state is not one that indicates a token has been found, then a routine **fail()** is invoked to retract the forward pointer to the position of the beginning pointer and to indicate a search for a token specified by the next transition diagram.
- If there are no other transition diagrams to try, **fail()** calls an error recovery routine.

- To return tokens we use a global variable **lexical_value**, which is assigned the pointers returned by functions **install_id()** and **install()_num** when an identifier or number, respectively, is found. The token class is returned by the main procedure of the lexical analyzer, called **nexttoken().**

We use a case statement to find the start state of the next transition diagram. In the C implementation, two variables state and start keep track of the present state and the starting state of the current transition diagram.

```
int state = 0, start = 0;
int lexical_value;
    /* to "return" second component of token */

int fail()
{
    forward = token_beginning;
    switch (start) {
        case 0:   start = 9; break;
        case 9:   start = 12; break;
        case 12:  start = 20; break;
        case 20:  start = 25; break;
        case 25:  recover(); break;
        default:   /* compiler error */
    }
    return start;
}
```

C code to find next start state

Edges in transition diagrams are traced by repeatedly selecting the code fragment for a state and executing the code fragment to determine the next state.

```
token nexttoken()
{   while(1) {
        switch (state) {
        case 0:   c = nextchar();
            /* c is lookahead character */
            if (c==blank || c==tab || c==newline) {
                state = 0;
                lexeme_beginning++;
                    /* advance beginning of lexeme */
            }
            else if (c == '<') state = 1;
            else if (c == '=') state = 5;
            else if (c == '>') state = 6;
            else state = fail();
            break;

            .../* cases 1-8 here */
```

```
            case 25:   c = nextchar();
                if  (isdigit(c)) state = 26;
                else state = fail();
                break;
            case 26:   c = nextchar();
                if (isdigit(c)) state = 26;
                else state = 27;
                break;
            case 27:   retract(1); install_num();
                return ( NUM );
            }
        }
    }
        case 9:    c = nextchar();
            if (isletter(c)) state = 10;
            else state = fail();
            break;
        case 10:   c = nextchar();
            if (isletter(c)) state = 10;
            else if (isdigit(c)) state = 10;
            else state = 11;
            break;
        case 11:   retract(1); install_id();
            return ( gettoken() );

            .../* cases 12-24 here */
```

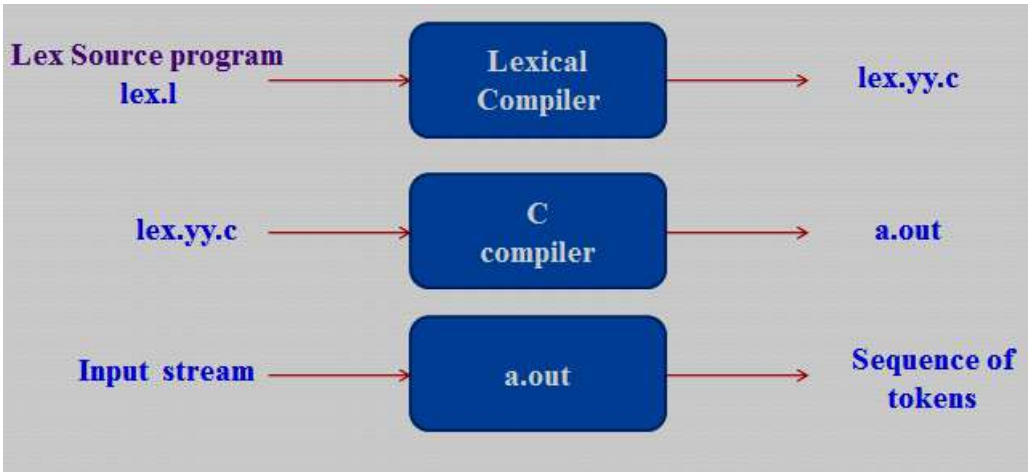
C code for lexical analyzer.

11. Elaborate on the language for specifying lexical analyzer. (6 MARKS) (NOV 2013)

- Several tools have been built for constructing lexical analyzers from special purpose notations based on regular expressions.
- The use of regular expressions for specifying tokens patterns.
- A particular tool called ***Lex,*** which is used to specify lexical analyzer for a variety of languages.
- We refer to the tool as the ***Lex compiler*** and to its input specification as the Lex language.

Creating a lexical analyzer with Lex

1. First, a specification of a lexical analyzer is prepared by creating a program ***lex.l*** in the Lex language.
2. Then, ***lex.l*** is run through the ***Lex compiler*** to produce a *C program* ***lex.yy.c.***
3. The program ***lex.yy.c*** consists of tabular representation of a transition diagram constructed from regular expression of ***lex.l***, together with a standard routine that uses the table to recognize lexemes.
4. The actions associated with regular expressions in *lex.l* are pieces of *C code* and are carried over directly to ***lex.yy.c.***
5. Finally ***lex.yy.c*** is run through the ***C compiler*** to produce an object program ***a.out***, which is the lexical analyzer that transforms an input stream into a sequence of tokens.



Lex Specifications

A Lex program consists of three parts:

declarations

%%

translation rules

%%

auxiliary procedures

- The *declarations section* includes declarations of variables, manifest constants, and regular definitions.
- The *translation rules* of a Lex program are statement of the form

  p_1 { $action_1$}

  p_2 { $action_2$}

  …… …..

  p_n { $action_n$}

  where each $\boldsymbol{p_i}$ is a regular expression and each $\boldsymbol{action_i}$ is a program fragment describing what action the lexical analyzer should take when pattern matches a lexeme.
- The third section holds whatever *auxiliary procedures* are needed by the actions. Alternatively, these procedures can be compiled separately and loaded with the lexical analyzer.

A lexical analyzer created by Lex behaves with a parser in the following manner.

- When activated by the parser, the lexical analyzer begins reading its remaining input, one character at a time, until it has found the longest prefix of the input that is matched by one of the regular expressions $\boldsymbol{p_i}$.
- Then it executes $\boldsymbol{action_i}$.
- The lexical analyzer returns a single quantity, the token, to the parser.
- To pass an attribute value with the information about the lexeme, we can set a global variable called ***yylval.***

Lex Program for the tokens

```
% {
        /* definitions of manifest constants
        LT, LE, EQ, NE, GT, GE,
        IF, THEN, ELSE, ID, NUMBER, RELOP */
% }

/* regular definitions */
delim           [  \t\n]
ws              {delim}+
letter          [A-Za-z]
digit           [0-9]
id              {letter}({letter}|{digit})*
number          {digit}+(\.{digit}+)?(E[+\-]?{digit}+)?

%%
{ws}            {/* no action and no return */}
if              {return(IF);}
then            {return(THEN);}
else            {return(ELSE);}
{id}            {yylval = install_id(); return(ID); }
{number}        {yylval = install_num(); return(NUMBER);}
"<"             {yylval = LT;  return(RELOP); }
"<="            {yylval = LE;  return(RELOP); }
"="             {yylval = EQ;  return(RELOP); }
"<>"            {yylval = NE;  return(RELOP); }
">"             {yylval = GT;  return(RELOP); }
">="            {yylval = GE;  return(RELOP); }
%%

install_id()
 {
        /* procedure to install the lexeme, whose first character is pointed to by yytext,
           and whose length is yyleng, into the symbol table and return a pointer  */
}
```

```
install_num()
{
        /* similar procedure to install a lexeme that is a number */
}
```

In the declaration section, the declaration of certain manifest constants used by the translation rules. These declarations are surrounded by the special brackets %{ and %}. Anything appearing between these brackets is copied directly into the lexical analyzer lex.yy.c and is not treated as part of the regular definitions or the translation rules.

The auxiliary procedures in the third section. There are two procedures, install_id and install_num, that are used by the translation rules; these procedures will be copied into lex.yy.c verbatim.

In the definitions section are some regular definitions. Each such definition consists of a name and a regular expression denoted by that name. For example, the first name defined is **delim**; it stands for the **character class [\t\n]**, that is any of the three symbols **blank**, **tab (\t)** or **newline (\n).** The second definition is of white space, denoted by the name **ws**. White space is any sequence of one or more delimiter character.

In the definition of letter, we see the use of a character class. The shorthand **[A-Za-z]** means any of the capital letters A through Z or the lower-case letters a through z. The fifth definition, of id, uses parentheses, which are metasymbols in Lex. Similarly, the vertical bar is Lex metasymbol representing union.

The translation rules in the section following the first **%%**. The first rule says that if we see ws, that is, any maximal sequence of blanks, tabs, and newlines, we take **no action**. In particular, we do not return to the parser.

The second rule says that if the letters **if** are seen, return the **token IF**, which is a manifest constant representing some integer understood by the parser to be the token if.

In the rule for id, we see two statements in the associated action. First, the variable **yylval** is set to the value returned by procedure **install_id**; the definition of that procedure is in the third section. **yylval** is a variable whose definition appears in the Lex output **lex.yy.c**, and which is also available to the parser. The purpose of **yylval** is to hold the lexical value returned, since the second statement of the action, **return (ID)**, can only return a code for the token class.

We may suppose that it looks in the symbol table for the lexeme matched by the pattern id. Lex makes the lexeme available to routines appearing in the third section through two variables **yytext** and **yyleng**. The variable **yytext** corresponds to the variable that we have been calling **lexeme_beginning**, that is, a pointer to the first character of the lexeme; **yyleng** is an integer telling how long the lexeme is. For example, if **install_id** fails to find the identifier in the symbol table, it might create a new entry for it. The **yyleng** characters of the input, starting at yytext, might be copied into a character array and delimited by an end of string marker. The new symbol table entry would point to the beginning of this copy.

Numbers are treated similarly by the next rule, and for the last six rules yylval is used to return is used to return a code for the particular relational operator found while the actual return value is the code for token **relop.**

12. Explain about Finite Automata? **(11 MARKS)** **(NOV 2016)**

A ***recognizer*** for a language is a program that takes as input a string x and answer "yes" if x is a sentence of the language and "no" otherwise. We compile a regular expression into a recognizer by constructing a generalized transition diagram called a finite automaton.

- An automata has a mechanism to read input from input tape,
- Any language is recognized by some automation, Hence these automation are basically language 'acceptors' or 'language recognizers'.

Types of Finite Automata

1. Deterministic Finite Automata (DFA)
2. Non-Deterministic Finite Automata (NFA)

A finite automaton can be deterministic on non-deterministic, where "non-deterministic" means that more than one transition out of a state may be possible on the same input symbol.

Both deterministic and non-deterministic finite automata are capable of recognizing precisely the regular sets. Thus they both can recognize exactly what regular expression can denote. DFA can lead to faster recognizers than NFA, a DFA can be much bigger than an equivalent NFA.

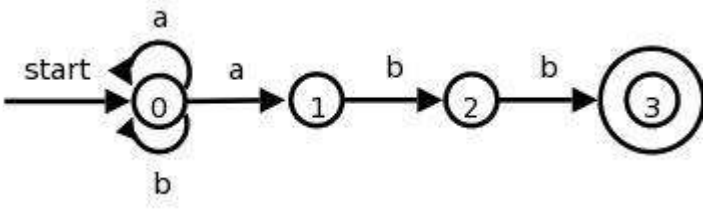
Non-Deterministic Finite Automata (NFA)

A NFA is a mathematical model that consists of

1. a set of states S
2. a set of input symbols Σ (the input symbol alphabet)
3. a transition for move from one state to another states.
4. A state s_0 that is distinguished as the start (or initial) state.
5. A set of states F distinguished as accepting (or final) state.

A NFA can be diagrammatically represented by a labeled directed graph, called a **transition graph**, in which the nodes are the states and the labeled edges represent the transition function. This graph looks like a transition diagram, but the same character can label two or more transitions out of one state and edges can be labeled by the special symbol € as well as by input symbols.

The transition graph for an NFA that recognizes the language (a|b)*abb is shown



Deterministic Finite Automata (DFA)

A deterministic finite automata has at most one transition from each state on any input. A DFA is a special case of a NFA in which:-

1. it has no transitions on input € ,
2. Each input symbol has at most one transition from any state.

DFA formally defined by 5 tuple notation M = (Q, Σ, δ, qo, F),

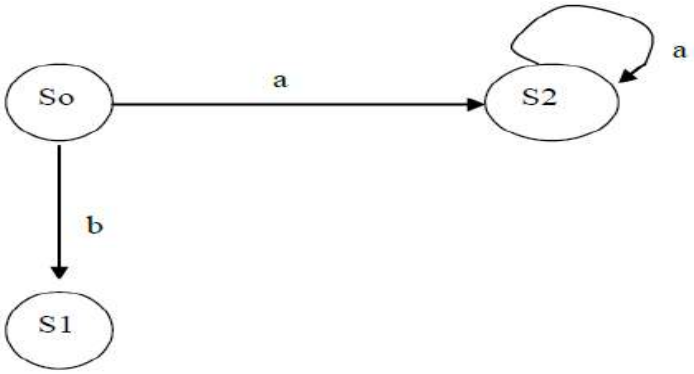
where

1. Q is a finite 'set of states', which is non empty.
2. Σ is 'input alphabets', indicates input set.
3. qo is an 'initial state' and qo is in Q ie, qo, Σ, Q
4. F is a set of 'Final states',
5. δ is a 'transmission function' or mapping function, using this function the next state can be determined.

The regular expression is converted into minimized DFA by the following procedure:

Regular expression → NFA → DFA → Minimized DFA

The Finite Automata is called DFA if there is only one path for a specific input from current state to next state.



From state S0 for input 'a' there is only one path going to S2. Similarly from S0 there is only one path for input going to S1.

Transition Table

- When describing an NFA, we use the transition graph representation. In computer, the transition function of an NFA can be implemented in several different ways. The easiest implementation in a transition table is which there is a row for each state and a column for each input symbol and €, if necessary.
- The transition table representation has the advantage that it provides fast access to the transitions of a given state on a given character; its advantage is that it can take up a lot of space when the input alphabet is large and most transitions are to the empty set.

13. Write a LEX program to count the vowels and consonants in a given string. (7 MARKS) (NOV 2017)

LEX Program to count the number of vowels and consonants in a given string:

Vow.l

```
%{
    int vow_count=0;
    int const_count =0;
%}

%%
    [aeiouAEIOU] {vow_count++;}
    [a-zA-Z] {const_count++;}
%%

int main()
{
    printf("Enter the string of vowels and consonants:");
    yylex();
    printf("The number of vowels are:  %d\n", vow);
    printf("The number of consonants are:  %d\n", cons);
    return 0;
}
```

Output:

```
$$ lex Vow.l
$$ gcc Vow.c -ll
$$ ./a.out
Enter the string of vowels and consonants:
My name is SMVEC
The number of vowels are: 4
The number of consonants are: 9
```

PONDICHERRY UNIVERSITY QUESTIONS

2 MARKS

1. What is hierarchical analysis? **(NOV 2011) (Ref.Qn.No.9, Pg.no.3)**
2. What is the major advantage of a lexical analyzer generator? **(NOV 2011) (Ref.Qn.No.31, Pg.no.8)**
3. List out the parts on Lex specifications. **(MAY 2012) (Ref.Qn.No.45, Pg.no.10)**
4. What is Compiler? **(MAY 2012, 2016) (NOV 2012) (Ref.Qn.No.1, Pg.no.2)**
5. What is meant by Static Checking? **(MAY 2014) (Ref.Qn.No.5, Pg.no.2)**
6. Define Sentinel. **(MAY 2014) (Ref.Qn.No.37, Pg.no.9)**
7. What is transition diagram? **(NOV 2012, 2016) (Ref.Qn.No.42, Pg.no.10)**
8. Why separate lexical analysis phase is required? **(MAY 2013) (Ref.Qn.No.30, Pg.no.7)**
9. State the function of front end and back end of a compiler phase. **(MAY 2013) (Ref.Qn.No.23,24 Pg.no.6)**
10. State with example the cousins of compilers. **(NOV 2013) (Ref.Qn.No.18, Pg.no.5)**
11. List the role of lexical analyzer? **(NOV 2013, 2015) (Ref.Qn.No.28, Pg.no.7)**
12. Compare and contrast Compilers and Interpreters. **(MAY 2015, 2017) (Ref.Qn.No.46, Pg.no.11)**
13. Give the structure of a Lex program with example. **(MAY 2015) (Ref.Qn.No.45, Pg.no.10)**
14. What is lexical analysis? **(MAY 2016) (Ref.Qn.No.8, Pg.no.3)**
15. What is meant by tokens? **(NOV 2016, 2017) (Ref.Qn.No.47, Pg.no.11)**
16. Define three address codes? **(NOV 2016) (Ref.Qn.No.14, Pg.no.4)**
17. Define Input Buffering. **(MAY 2017) (Ref.Qn.No.36, Pg.no.9)**
18. Write regular expression for the following language over the alphabet over the alphabet $\sum$ = {a, b}.
    All strings that contain an even number of b's. **(NOV 2017) (Ref.Qn.No.48, Pg.no.11)**
19. What is a code generator? **(MAY 2018) (Ref.Qn.No.16, Pg.no.4)**
20. What will be the output for the lexical analyzer? **(MAY 2018) (Ref.Qn.No.28, Pg.no.7)**

11 MARKS

NOV 2011 (REGULAR)

1. Draw the different phases of a compiler and explain. **(Ref.Qn.No.3, Pg.no.17)**

(OR)

2. How to recognize the tokens? **(Ref.Qn.No.10, Pg.no.35)**

MAY 2012 (ARREAR)

1. Explain the Cousins of the compiler. **(Ref.Qn.No.4, Pg.no.23)**

(OR)

2. Discuss the Phases of a compiler. **(Ref.Qn.No.3, Pg.no.17)**

NOV 2012 (REGULAR)

1. Explain the phases of a compiler. **(Ref.Qn.No.3, Pg.no.17)**

(OR)

2. Discuss the role of the lexical analyzer. **(Ref.Qn.No.7, Pg.no.28)**

MAY 2013 (ARREAR)

1. a) State the different compiler construction tools and their use. (6) **(Ref.Qn.No.5, Pg.no.26)**
   b) Illustrate the steps involved in the recognition of tokens. (5) **(Ref.Qn.No.10, Pg.no.35)**

(OR)

2. With a neat sketch discuss the functionalities of various phases of a compiler. **(Ref.Qn.No.3, Pg.no.17)**

NOV 2013 (REGULAR)

1. Describe the different stage of a compiler with an example. Consider an example for a simple arithmetic expression statement. **(Ref.Qn.No.3, Pg.no.17)**

(OR)

2. Explain the buffered I/O with sentinels. Elaborate on the language for specifying lexical analyzer. **(Ref.Qn.No.8, Pg.no.30) (Ref.Qn.No.11, Pg.no.40)**

MAY 2014 (ARREAR)

1. Discuss about Phases of compiler in compiler structure? **(Ref.Qn.No.3, Pg.no.17)**

(OR)

2. Explain in detail how to handle lexical errors due to transposition of the letters. **(Ref.Qn.No.7, Pg.no.28)**

NOV 2014 (REGULAR)

1. Explain Phases of a compiler with neat diagram. **(Ref.Qn.No.3, Pg.no.17)**

(OR)

2. What is the role of the lexical analyzer? Explain **(Ref.Qn.No.7, Pg.no.28)**

MAY 2015 (ARREAR)

1. Discuss in detail various phases of compilation with a neat diagram. **(Ref.Qn.No.3, Pg.no.17)**

(OR)

2. Convert the following regular expression to equivalent ε-NFA and then to DFA.
   (0|1)* 0 (0|1)

NOV 2015 (REGULAR)

1. Describe the Architecture of Transition Diagram based lexical Analyzer. **(Ref.Qn.No.10, Pg.no.35)**

(OR)

2. Write short notes on
   (a). Semantic Analysis. (5) **(Ref.Qn.No.2, Pg.no.16)**
   (b). Compiler construction tools. (6) **(Ref.Qn.No.6, Pg.no.26)**

MAY 2016 (ARREAR)

1. Write the steps required to translate the source program to object program. **(Ref.Qn.No.3, Pg.no.17)**

(OR)

2. Give a detailed account on lexical analysis. **(Ref.Qn.No.7, Pg.no.28)**

NOV 2016 (REGULAR)

1. Explain about finite automata. **(Ref.Qn.No.12, Pg.no.45)**

(OR)

2. Describe in detail about the phases of compiler. **(Ref.Qn.No.3, Pg.no.17)**

MAY 2017 (ARREAR)

1. Explain in detail about Compiler Construction tools. **(Ref.Qn.No.6, Pg.no.26)**

(OR)

2. Elaborate in detail about Design of Lexical Analyser Generator. **(Ref.Qn.No.7, Pg.no.28)**

NOV 2017 (REGULAR)

1. (a). Write a LEX program to count the vowels and consonants in a given string. **(7)** **(Ref.Qn.No.13, Pg.no.46)**

   (b). Discuss the cousins of compiler. **(4)** **(Ref.Qn.No.4, Pg.no.23)**

   (OR)

2. (a). Discuss the different phases of a compiler. **(7)** **(Ref.Qn.No.3, Pg.no.17)**

   (b). List and explain any 4 regular-expression construction rules. **(4)** **(Ref.Qn.No.6, Pg.no.26)**

MAY 2018 (ARREAR)

1. Write a neat diagram, explain the phases of a compiler. **(Ref.Qn.No.3, Pg.no.17)**

   (OR)

2. List out the role of lexical analyzer. How tokens are recognized in the source program? Explain it with an example. **(Ref.Qn.No.7,10, Pg.no.28,35)**